Build-From-Scratch Systems Portfolio

Product Requirements Document

Author: Abhinav | Status: Draft v1

Purpose: Define seven independent software projects built from first principles to learn system design and strengthen a resume and portfolio.

---

1. Overview

This document describes a portfolio of systems projects. Each one starts from a real problem that an existing, well-known tool solves. We study the problem, then design our own solution in our own way, with a deliberately smaller scope and a simpler technology stack.

1.1 Guiding Principles

1. Problem first, not copy first. Each project begins with the problem it solves, not with the original product's source code or internals.
2. Our own design. We choose our own data formats, protocols, algorithms and languages. We do not aim for byte-for-byte or API compatibility with the originals. Compatibility may be added later as an optional stretch goal.
3. Functional, not exhaustive. Each project delivers a small set of features that work reliably, rather than a long list of half-working ones.
4. Measure everything. Every project ships with tests and benchmarks, so claims on a resume are backed by numbers.
5. Document the thinking. Every project includes a short design document covering trade-offs and what was left out.

1.2 Project Summary

#	Project	Problem Area	Inspired By (problem only)
1	Compression Engine	Storage and bandwidth cost	gzip, zip
2	In-Memory Key-Value Store	Fast access to hot data	Redis
3	Version Control System	Tracking and merging changes	Git
4	Code Hosting Platform	Collaboration on shared code	GitHub
5	Encryption Toolkit	Confidentiality and integrity	AES, SHA family
6	Reliable Phone-to-Windows Sharing	Unreliable cross-device data transfer	Quick Share, Phone Link
7	Phone Desktop Workspace	Computer-like experience on any phone	Samsung DeX

1.3 Suggested Build Order

Compression first (small, self-contained, feeds Projects 3 and 6). Then the key-value store and version control system. Encryption runs alongside. The code hosting platform comes after the version control system. Project 6 can start any time after compression and encryption. Project 7 comes last because it depends heavily on device hardware and Android limits.

1.4 Common Deliverables for Every Project

- Public repository with a README (problem, architecture diagram, setup, demo, limitations)

- Automated tests
- Benchmark results in a table
- A one-page design note: decisions, trade-offs, non-goals
- A portfolio case-study page and, once finished, a resume bullet

1.5 Technology Choices

Language is chosen per project, based on learning goals. Native languages such as C++ suit byte-level work (Projects 1 and 5). Node.js, Java or similar suit servers and web apps (Projects 2, 3, 4, 6). Final choices are recorded in each project's design note.

2. Project 1: Compression Engine

2.1 Problem Statement

Data is expensive to store and slow to send. Text, logs, JSON and documents contain a lot of repetition, and without compression that repetition wastes disk space and network bandwidth. People need a way to shrink data and restore it exactly, without losing a single byte.

2.2 Goals

- Reduce the size of common file types measurably.
- Restore files exactly (lossless).
- Be fast enough to use on ordinary laptops.
- Serve as a reusable library for Projects 3 and 6.

2.3 Our Approach

We design our own container format and our own combination of techniques, guided by first principles: finding repeated patterns and assigning shorter codes to frequent symbols. We experiment with several strategies, compare them, and keep the best, instead of reproducing an existing standard.

2.4 Core Features (v1)

- Compress and decompress a single file from the command line
- Our own file format with a header, version number and integrity checksum
- At least two compression strategies, selectable or chosen automatically
- Progress output and clear error messages for corrupted input
- A library interface other projects can call

2.5 Stretch Features

- Streaming mode for large files
- Folder and multi-file archives
- Multi-threaded compression

2.6 Non-Goals

Lossy compression (images, audio, video), and compatibility with existing formats such as zip or gzip.

2.7 Success Metrics

- 100% round-trip correctness on a test corpus (text, JSON, source code, binaries, empty files, random data)
- Compression ratio and speed reported against common tools on the same corpus
- Corrupted files are always detected, never silently accepted

2.8 Risks

Random or already-compressed data cannot be shrunk and may grow slightly. The tool must handle this gracefully, for example by storing such data uncompressed.

3. Project 2: In-Memory Key-Value Store

3.1 Problem Statement

Applications repeatedly need the same data, such as sessions, counters and cached results. Reading it from a disk-based database every time is slow and puts load on the database. Developers need a very fast, simple place to keep frequently used data in memory, with optional expiry and optional durability.

3.2 Goals

- Sub-millisecond reads and writes for typical data.
- Simple, easy-to-learn commands.
- Support multiple simultaneous clients.
- Optional persistence so data survives a restart.

3.3 Our Approach

We design our own client-server protocol and our own command set, instead of reproducing an existing one. The server concentrates on a clean in-memory data model, predictable concurrency, and simple, understandable persistence.

3.4 Core Features (v1)

- TCP server handling many clients
- Set, get and delete values by key
- Time-to-live (expiry) on keys
- Counters and at least two additional data types (for example lists and maps)
- Persistence through a change log that is replayed on startup
- A small command-line client

3.5 Stretch Features

- Periodic snapshots to speed up restarts
- Publish/subscribe messaging
- Memory limit with an eviction policy
- Basic password authentication

3.6 Non-Goals

Clustering, replication, scripting, and compatibility with any existing client library.

3.7 Success Metrics

- Operations per second and latency (average and 99th percentile) under a defined load
- Data fully recovered after a forced shutdown
- Correct expiry behavior verified by tests
- Stable with at least 100 concurrent clients

3.8 Risks

Concurrency bugs and unbounded memory growth. Mitigation: a clear concurrency model chosen up front, plus load tests.

4. Project 3: Version Control System

4.1 Problem Statement

Without version control, people save copies like "final_v2_final_REAL", overwrite each other's work, and cannot answer who changed what, when and why. Teams need a way to record the history of a project, return to any earlier state, work on separate lines of development, and combine them safely.

4.2 Goals

- Record project history reliably and compactly.
- Let users go back to any saved point.
- Support parallel lines of work and combining them.
- Detect corruption.

4.3 Our Approach

We design our own storage layout and our own way of identifying and linking snapshots. We use the compression engine (Project 1) for storage efficiency and focus on a clear, well-tested model of snapshots, history and branches.

4.4 Core Features (v1)

- Initialize a repository
- Stage and save snapshots with a message and author
- View history
- Show what changed (status and differences between versions)
- Create, list and switch lines of development (branches)
- Restore earlier versions
- Ignore rules for files that should not be tracked

4.5 Stretch Features

- Combine two branches, with conflict detection
- Tags for named versions
- Sync changes between two repositories (feeds Project 4)
- Optional compatibility with existing repository formats

4.6 Non-Goals

Performance parity with mature tools on huge repositories, and advanced history rewriting.

4.7 Success Metrics

- Round-trip test: save, change, restore gives identical files
- Repository size compared with plain file copies for a sample project
- Integrity check detects any deliberately corrupted file
- Combining branches gives correct results on a defined set of test scenarios

4.8 Risks

Merge logic is the hardest part. Mitigation: ship history and branching first, merging later.

5. Project 4: Code Hosting Platform

5.1 Problem Statement

A version control system works on one machine. Teams need a shared, always-available home for repositories, with accounts, permissions, a way to browse code and history in a browser, and a structured way to propose, discuss and approve changes before they are merged.

5.2 Goals

- Central hosting for repositories from Project 3.
- Simple collaboration: review and discuss before merging.
- Usable from a web browser without installing anything.

5.3 Our Approach

A small web application with its own account system, repository storage, and its own sync protocol between client and server. We prioritize a clear core flow (sign up, create a repository, push, browse, propose changes, review, merge) over a long feature list.

5.4 Core Features (v1)

- User registration, login and sessions
- Create public and private repositories
- Push and pull changes
- Browse files, folders, history and differences in the browser
- Issues with comments
- Change proposals (pull-request style) with review comments and merge
- Access control: owner and collaborator roles

5.5 Stretch Features

- Notifications
- Search across repositories
- Webhooks
- Encrypted private repositories (using Project 5)
- Simple activity dashboard

5.6 Non-Goals

Continuous integration, package registries, project boards, and large-scale hosting.

5.7 Success Metrics

- Complete core flow works end to end in an automated test
- Private repositories are inaccessible to non-collaborators (security tests)
- Page load time under an agreed target on a typical repository

5.8 Risks

Security (authentication, authorization, input handling) and scope creep. Mitigation: security checklist per feature, and a strict v1 scope.

6. Project 5: Encryption Toolkit

6.1 Problem Statement

Files and messages stored or sent over networks can be read or altered by anyone who gets access to them. People need a way to keep data confidential, to know it has not been tampered with, and to store passwords safely.

6.2 Goals

- Encrypt and decrypt files and data with a password or key.
- Detect tampering.
- Store passwords in a way that resists theft.
- Provide building blocks for Projects 4 and 6.

6.3 Our Approach

We build the toolkit around our own file format and key-handling design, and implement the underlying algorithms ourselves to understand how they work.

> **Important safety note.** Security algorithms are not a place for inventing new designs. A homemade cipher that looks strong can be broken easily, and even correct implementations of standard algorithms can leak secrets through subtle bugs. Therefore:
>
> • Any algorithm we write ourselves is for learning and demonstration only, and the README must say so clearly.
> • We will not use it to protect real personal, customer or business data.
> • Where real protection is needed (for example in Projects 4 and 6), we use a vetted, well-reviewed library for the actual cryptography and keep our own implementation separate as a learning artifact.
> • Our own work is validated against official published test vectors wherever they exist.

6.4 Core Features (v1)

- Encrypt and decrypt a file with a password
- Key derivation from passwords with salting
- Integrity check (tamper detection) on every decrypted file
- Secure password hashing for account storage
- Command-line interface and a library interface

6.5 Stretch Features

- Streaming encryption for large files
- Key exchange demonstration between two parties
- Digital signatures demonstration

6.6 Non-Goals

Being a replacement for audited cryptographic libraries, and certification or compliance claims.

6.7 Success Metrics

- Matches official test vectors for each implemented primitive
- Any modification of an encrypted file is detected on decryption
- Wrong password or key always fails cleanly
- Documented threat model: what we protect against and what we do not

6.8 Risks

Implementation mistakes and false confidence. Mitigation: test vectors, a documented threat model, and the safety note above in every README.

7. Project 6: Reliable Phone-to-Windows Data Sharing

7.1 Problem Statement

There is no dependable way to move data between a phone and a Windows computer. USB file transfer is inconsistent, wireless tools often fail to find the device (for example when a router isolates devices or a firewall blocks discovery), Bluetooth is slow, and most tools depend on a single connection method, so when it fails the transfer fails. Users end up emailing files to themselves or uploading them to the cloud.

7.2 Goals

- Transfers that complete reliably, even over unreliable connections.
- Works across different phones, networks and Windows setups.
- No account or internet connection required.
- Simple enough for non-technical users.

7.3 Our Approach

Treat reliability as the product. Our design uses multiple connection methods with automatic fallback, splits files into verified pieces so transfers can resume, and pairs devices through a simple scan-to-connect step instead of depending on network discovery.

7.4 Core Features (v1)

- Android app and Windows app
- Pairing by QR code
- Send and receive files in both directions
- At least two connection methods with automatic fallback (for example same network, then phone hotspot)
- Chunked transfers with integrity checks and resume after interruption
- Transfer progress, speed, and clear failure messages
- Encrypted sessions

7.3.1 Research Step (before building)

Collect real failure reports from users and test existing tools on several phone and network setups, so we design against actual failure modes.

7.5 Stretch Features

- Folder transfer
- Clipboard and link sharing
- Optional compression of transfers
- Transfer over USB cable as a third method
- Transfer history

7.6 Non-Goals

iPhone support in v1, cloud relay, and remote access over the internet.

7.7 Success Metrics

- Transfer success rate across a test matrix of phones, networks and Windows versions
- Interrupted 1 GB transfer resumes without restarting

- Speed compared with existing tools on the same network
- Fallback activates correctly when the primary connection is blocked

7.8 Risks

Variation across Android versions and manufacturer customizations, and Windows firewall behavior. Mitigation: a test device matrix and clear diagnostics shown to the user.

8. Project 7: Phone Desktop Workspace

8.1 Problem Statement

A modern phone is as powerful as a basic computer, but the experience is limited to a small touchscreen. Samsung DeX offers a desktop experience when connecting a phone to a monitor with a keyboard and mouse, but it is limited to certain Samsung devices. Users with other phones, especially budget ones, have no equivalent, even though many of them already own a monitor and a keyboard.

8.2 Goals

- Give users a computer-like workspace on a larger screen using their existing phone.
- Adapt the layout to different screen sizes and connection types.
- Work well with a keyboard and mouse.

8.3 Our Approach

An Android application that acts as a desktop-style environment: its own taskbar, launcher and resizable panels, with built-in productivity tools and an adaptive layout engine that reflows for phone, tablet and monitor sizes. We work within what Android allows apps to do and report honestly which devices are supported.

8.4 Core Features (v1)

- Detect an external display and switch into workspace mode
- Desktop-style home screen with taskbar and launcher
- Keyboard and mouse support: pointer, scrolling, shortcuts, right-click
- Built-in tools: file manager, document or PDF viewer, notes and a simple web view
- Adaptive layout that scales across screen sizes and resolutions
- A compatibility checker that tells users whether their phone can output video and how

8.5 Stretch Features

- Floating, resizable windows for built-in tools
- Wireless display support
- Integration with Project 6 for moving files between phone and PC
- Multiple workspaces

8.6 Non-Goals

Running all third-party apps in freely resizable windows (restricted by the operating system on most phones), custom firmware, and adding hardware video output to phones that lack it.

8.7 Success Metrics

- Tested on a defined list of phones, with a published compatibility table
- Layout stays usable at common screen sizes and resolutions
- Input latency for keyboard and mouse stays within an acceptable target

8.8 Risks

Hardware limits (many budget phones cannot output video), Android restrictions on window management, and device fragmentation. Mitigation: validate device support early with a minimal prototype before committing to the full feature set.

9. Cross-Project Dependencies

- Project 1 (compression) is used by Projects 3 and 6.
- Project 5 (encryption) is used by Projects 4 and 6.
- Project 3 (version control) is required by Project 4.
- Project 6 can feed file transfer into Project 7.

10. Portfolio and Resume Plan

- Create one repository per project once a working prototype exists.
- Mark unfinished projects as "In progress" and list only finished work on the resume.
- Write one case-study page per project on the portfolio website covering problem, design decisions, trade-offs and results.
- Share a short write-up on LinkedIn at each completed milestone.

11. Open Questions

1. Which language will each project use? (Decide per project before starting.)
2. Should Project 3 stay fully custom, or add optional compatibility with existing repository formats later?
3. Which phones and Windows versions will form the Project 6 and 7 test matrix?
4. How much time per week is available? (Needed to turn the build order into dates.)